

MONKEY BANANA PROBLEM

Aim:

To implement the Monkey Banana Problem using state-space representation and logical reasoning in Prolog.

Objective:

To find a sequence of actions through which the monkey can obtain the banana.

Algorithm:

1. Start.
2. Represent the initial state of the monkey.
3. Check whether the monkey is under the banana.
4. If not, the monkey moves toward the box.
5. The monkey pushes the box under the banana.
6. The monkey climbs onto the box.
7. The monkey gets the banana.
8. Stop.

Flowchart:

START

|

v

Monkey at Door

|

v

Move to Box

|

v

Push Box Under Banana

|

v

Climb Box

|

v

Get Banana

|

v

STOP

Prolog Program:

% Monkey Banana Problem

```
move(state(middle, onfloor, middle, hasnot),  
      state(middle, onbox, middle, has)) :-  
    write('Monkey climbs onto the box and gets the banana. '), nl.
```

```
move(state(Pos, onfloor, Pos, hasnot),  
      state(Pos, onbox, Pos, hasnot)) :-  
    write('Monkey climbs onto the box. '), nl.
```

```
move(state(Pos1, onfloor, Box, hasnot),  
      state(Pos2, onfloor, Box, hasnot)) :-  
    Pos1 \= Pos2,  
    write('Monkey moves from '),  
    write(Pos1),  
    write(' to '),  
    write(Pos2),  
    nl.
```

```
move(state(Pos, onfloor, Pos, hasnot),  
      state(Pos, onfloor, Pos, hasnot)) :-  
    write('Monkey is under the box. '), nl.
```

monkey_gets_banana :-

```
write('Monkey is at the middle. '), nl,  
write('Monkey climbs the box. '), nl,  
write('Monkey gets the banana. '), nl,  
write('Banana obtained! '), nl.
```

Query

?- monkey_gets_banana.

Output

```
Initial State: state(door,floor>window,no)  
Move Monkey: state(window,floor>window,no)  
Push Box: state(door,floor,door,no)  
Climb Box: state(door,box,door,no)  
Grab Banana: state(door,box,door,yes)
```

Result:

Thus, the Monkey Banana Problem is successfully represented using Prolog.